

Architecture Blueprint for a New Flutter + FastAPI Application

A production-grade, officially-grounded reference for a server-side architect building a first Flutter (Android, iOS, Web) frontend against a FastAPI backend, optimized for maintainability, scalability, module boundaries, and AI-assisted development that consistently respects the architecture.

Executive Summary



Adopt the Flutter team's official MVVM architecture

The Flutter team publishes a formal "Guide to app architecture" and a walk-through Compass sample app on docs.flutter.dev. It defines UI (View + ViewModel), optional Domain (use-cases), and Data (Repositories + Services) with strict rules for who talks to whom. This is exactly the kind of officially blessed structure a server-side architect can trust.



State management: pick one that matches the official model

The Flutter guide uses `ChangeNotifier` + `package:provider` as its canonical example, but treats state management as "Conditional" and endorses others. In 2026, the two mature production choices are Riverpod 3 and Bloc/Cubit; for a solo developer starting fresh with server-side habits, Bloc/Cubit maps most naturally to the official MVVM guide.



Use Flutter's official AI channels to enforce architecture

The team ships three complementary AI mechanisms on `docs.flutter.dev/ai`: a downloadable `rules.md` template, the official `dart mcp-server`, and the flutter/agent-plugins Agent Skills (including `flutter-apply-architecture-best-practices`). Wiring these into Claude Code/Cursor/Copilot removes most repeated "deep-review" cycles.



Mirror the same modular thinking on the FastAPI backend

The official FastAPI "Bigger Applications" tutorial and the community's most-starred zhanymkanov/fastapi-best-practices both recommend a per-domain folder (`router.py` , `schemas.py` , `models.py` , `service.py` , `dependencies.py` , `exceptions.py`). This aligns 1:1 with the Flutter feature layout.



Migration path: greenfield the shell, port features one at a time

Because the recommended structure changes both the folder layout and the widget contracts, a big-bang rewrite is risky. The lower-risk path is: create a new project with the Compass-style skeleton + rules + MCP + skills, then port existing features into it one `ui/<feature>` module at a time, with tests migrated first.

Bottom line: don't invent an architecture. Adopt the one published on `docs.flutter.dev` that is codified by the Flutter team, illustrated by the official Compass sample app, and reinforced by the Flutter team's own AI rules/MCP/skills. Then align the FastAPI backend with the officially recommended "bigger applications" structure and the widely used zhanymkanov/fastapi-best-practices conventions. The blueprint at the end of this report is a direct assembly of those sources.

1. Official Flutter Architecture: What the Flutter Team Actually Recommends

1.1 There is a formal, official architecture guide

The Flutter team maintains a first-class "**Architecting Flutter apps**" section under docs.flutter.dev/app-architecture. It explicitly targets scalable, multi-developer, feature-rich apps and states: *"This is the recommended way to build a Flutter app."* [20](#)

Its stated goals map exactly to the user's priorities — **maintainability, scalability, testability, lower cognitive load** [20](#) — which is precisely why grounding the project in this guide (rather than a custom Clean-Architecture variant) is the right decision for a server-side architect new to Flutter.

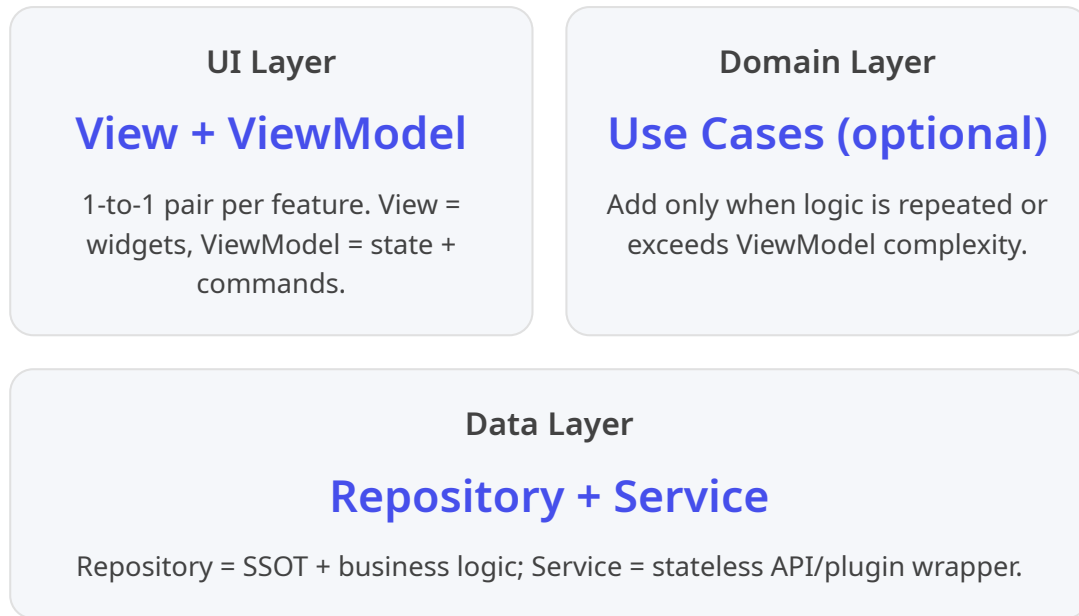
1.2 The core principles the whole design is built on

The "Common architecture concepts" page defines five non-negotiable principles that the entire guide (and this report) enforce: [19](#)

1. **Separation of concerns** — split into layers; within each layer, further separate by feature. [19](#)
2. **Layered architecture** — 2 to 3 layers (UI, optional Logic/Domain, Data); each layer only communicates with adjacent layers. [19](#)
3. **Single source of truth (SSOT)** — every data type has exactly one class (typically a Repository) that owns and mutates it. [19](#)
4. **Unidirectional data flow** — state flows Data → Logic → UI; events flow UI → Logic → Data. [19](#)
5. **UI is a function of (immutable) state** — data drives the UI; views hold as little logic as possible. [19](#)

Two secondary principles — extensibility (clean input/output contracts) and testability (easy mocking) — fall out of the five above. [19](#)

1.3 The layer model: MVVM, plus an optional Domain layer



The **Guide to app architecture** page lays this out formally: [18](#)

- **Views** — widget compositions, no business logic beyond simple conditionals, animations, layout-by-screen-size, and simple routing. [18](#)
- **View models** — hold the logic that turns repository data into UI state, expose commands (using the Command pattern) for user events, and are one-to-one with a view. [18](#)
- **Repositories** — the source of truth for a domain data type; handle caching, retry, refresh, polling, and expose domain models to the UI. Repositories are never aware of each other; combining data from two repositories is done in the ViewModel or a use-case. [18](#)
- **Services** — stateless wrappers around a single external data source (REST API, iOS/Android platform, local file). One service per data source. [18](#)
- **Use-cases (optional Domain layer)** — only added when a ViewModel would otherwise become complex or when logic is shared across ViewModels. The guide is explicit about the pros/cons and recommends adding them only when needed. [18](#)

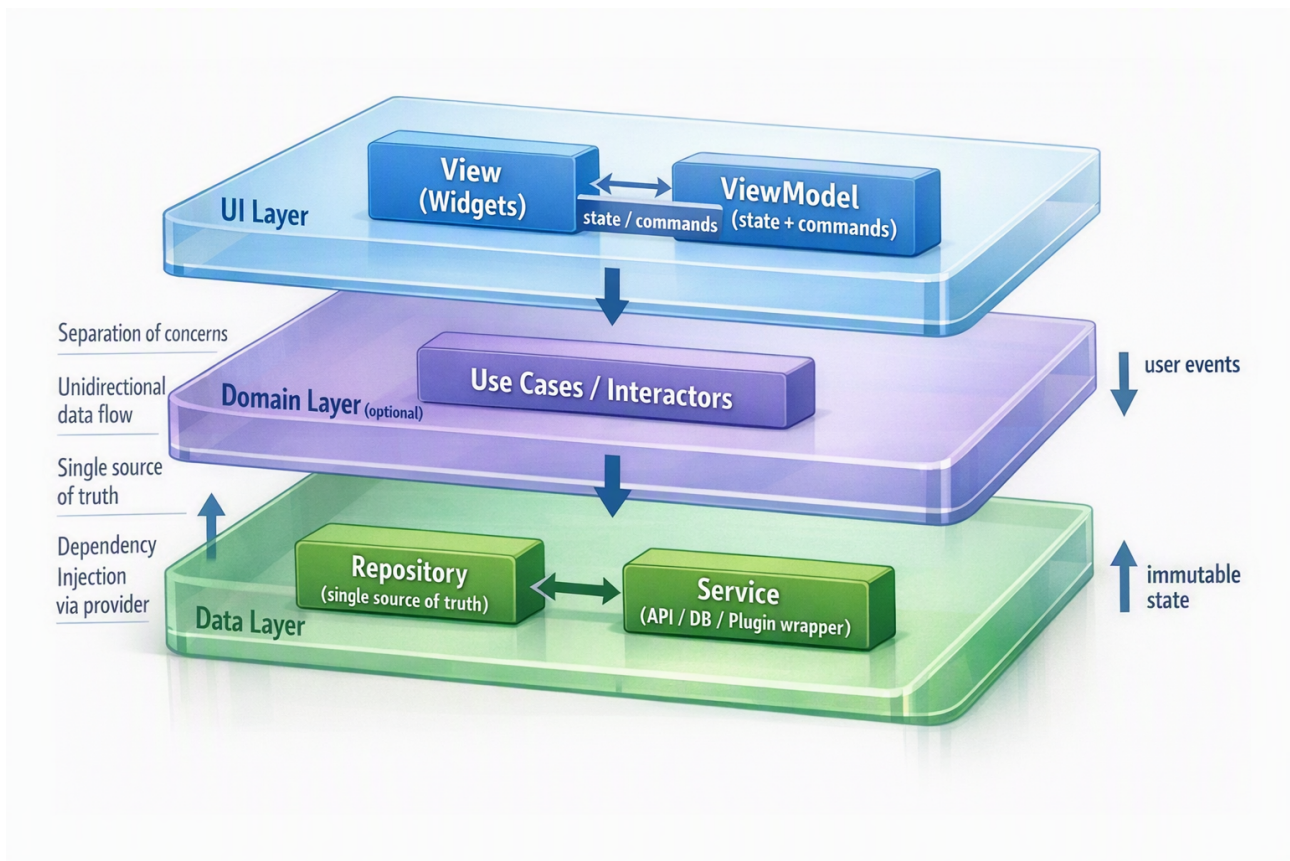


Figure 1: The official Flutter MVVM architecture — a UI layer of Views and ViewModels, an optional Domain layer of use-cases, and a Data layer of Repositories and Services, with immutable state flowing up and user events flowing down.

1.4 The strict rules of engagement between components

The **Communicating between layers** page defines a rule table that eliminates most "should this go here?" questions: [16](#)

Component	Rule
View	Aware of exactly one ViewModel; nothing else.
ViewModel	Belongs to exactly one View; can depend on one or more Repositories (via constructor).

Component	Rule
Repository	Can depend on many Services (via constructor); does not know Views or ViewModels.
Service	Depends on nothing else in the app.

Communication is done by **constructor injection** — dependencies are private fields, so a View can never bypass its ViewModel to call a Repository directly. [16](#) The Flutter team's official DI package is `package:provider`: services and repositories are exposed via `MultiProvider` at the top of the widget tree, and ViewModels are typically constructed inside `go_router` route builders where the required repositories are read from context. [16](#)

1.5 Recommended package structure (from the Compass case study)

The **Architecture case study** page shows how the official Compass sample app combines *feature-first* (in the UI layer) with *type-first* (in the Data layer). The layout is: [17](#)

```
lib/
  ui/
    core/           # shared widgets + themes
    ui/
    themes/
  <feature>/
    view_models/
      <feature>_viewmodel.dart
    widgets/
      <feature>_screen.dart
  domain/
    models/         # pure Dart domain models
  data/
    repositories/
    services/
    model/          # raw API models
  config/
  utils/
  routing/
  main.dart
  main_development.dart
  main_staging.dart
  test/             # mirrors lib/
    data/
```

```

domain/
ui/
utils/
testing/           # subpackage with fakes + shared mocks
fakes/
models/

```

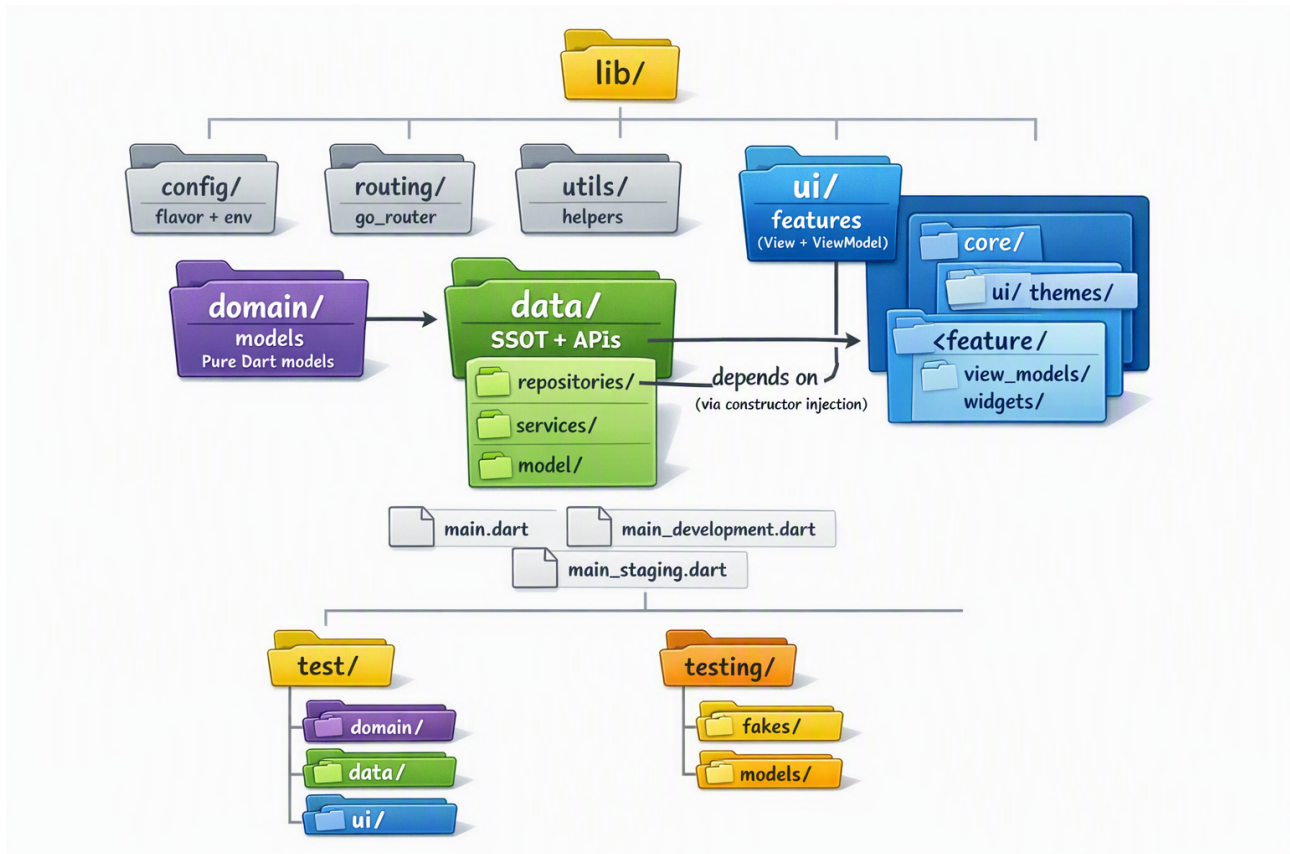


Figure 2: The recommended Flutter project structure derived from the official Compass sample app — features organized inside the UI layer, data organized by type, and a mirroring test tree with a dedicated testing package for shared fakes.

This layout is important for two reasons the guide highlights: [17](#)

- **Data is not tied to a single feature**, so grouping it by type (repositories/, services/) prevents duplicate repositories and makes the SSOT rule easy to enforce.
- **UI is grouped by feature**, so adding or deleting a feature is a single-folder operation, and the widget-to-ViewModel 1:1 relationship stays visually obvious.

The Compass repo confirms this structure in code: the top level of `compass_app/app/lib/` is exactly `config/`, `data/`, `domain/`, `routing/`, `ui/`, `utils/`, plus the three flavored entry points. [11](#)

1.6 Design patterns the guide bakes in

Beyond the layer model, docs.flutter.dev/app-architecture/design-patterns promotes five reusable patterns that make the architecture robust: ¹³

1. **Optimistic state** — improve perceived performance.
2. **Persistent storage: key-value** — for app config/preferences.
3. **Persistent storage: SQL** — for structured local data.
4. **Offline-first support** — repository pattern that mixes local + remote.
5. **The Command pattern** — wraps a ViewModel method with running/complete/error states, so views can render loading and error UI without extra flags.
6. **Result objects for error handling** — replaces unchecked exceptions between layers with an explicit `Result` type.

Adopting the Command pattern and Result objects from day one is one of the highest-leverage things the report recommends, because it eliminates a whole class of "why is the UI stuck?" bugs when moving from server-side code to declarative UI.

1.7 The official recommendations table (with priorities)

docs.flutter.dev/app-architecture/recommendations lists every recommendation with a priority: **Strongly recommend**, **Recommend**, or **Conditional**. ¹⁴ The critical ones to internalize: ¹⁴

- **Strongly recommend:** clearly defined data + UI layers; repository pattern; ViewModels + Views (MVVM); no logic in widgets; unidirectional data flow; immutable data models; dependency injection; abstract repository classes; unit-test every repository/service/ViewModel; use fakes.
- **Recommend:** `Command` objects for user events; `freezed` or `built_value` for immutable models; `go_router` for navigation; standardized naming conventions.
- **Conditional:** `ChangeNotifier` / `Listenable` for widget updates ("*ultimately the decision comes down to personal preference*"); an explicit Domain layer; separate API models vs. domain models (only in large apps).

The **Conditional** rating on `ChangeNotifier` is the door the Flutter team leaves open for Bloc, Riverpod, and others — the guide even links to the "other popular options" page. ¹⁴

2. State Management: A Comparison and Recommendation

2.1 What the Flutter docs themselves say

The state-management docs page explicitly refuses to name a single winner: it lists built-in tools (`setState` , `ValueNotifier` / `InheritedNotifier` , `InheritedWidget`) and points to [pub.dev](#) 's `#state-management` topic for community packages. ⁷ So the "official" answer is: any of the mature options is acceptable, provided you honor the MVVM/SSOT/UDF principles.

That said, the **case study uses** `package:provider` + `ChangeNotifier` because the Flutter team wanted the canonical example to depend only on packages they themselves recommend for DI. ¹⁶

2.2 Comparison of the three mainstream options in 2026

Dimension	Provider + ChangeNotifier	Bloc / Cubit	Riverpod 3
Maintained by	Flutter team (DI) + Remi Rousselet	Felix Angelov	Remi Rousselet
Paradigm	ChangeNotifier + observers	Event → State machine (or Cubit: method → State)	Global providers, reactive graph
Boilerplate	Low	High (Bloc) / Medium (Cubit)	Very low (with code-gen)

Dimension	Provider + ChangeNotifier	Bloc / Cubit	Riverpod 3
Learning curve	Easy	Steep initially; predictable long-term 24	Easy syntactically; discipline required to avoid "spaghetti providers" 24
Compile-time safety	Runtime lookup, can throw <code>ProviderNotFoundException</code>	Strong (typed events/states, especially with <code>freezed</code>)	Strong (compile-time safe, especially with <code>riverpod_generator</code>) 24
Testability	Good	Excellent (<code>bloc_test</code> asserts state sequences) 24	Excellent (<code>ProviderContainer</code> with overrides) 24
Code generation	None	Optional (<code>freezed</code> for events/states)	First-class (<code>@riverpod</code> annotation) 24
Alignment with docs.flutter.dev	Native — used in the Compass case study	Layered ("Presentation / Business Logic / Data") maps 1:1 to Flutter's UI/ Logic/Data; also the layout used by the official Very Good CLI 6 4	Different mental model (providers as global graph); requires care to keep View↔ViewModel 1:1

Dimension	Provider + ChangeNotifier	Bloc / Cubit	Riverpod 3
Ecosystem maturity	Very high (widely used, still supported)	Very high (enterprise standard) 24	Very high; documentation matured in 2025-2026 24
Fit for a solo dev from a server-side background	Gentlest entry, but you'll outgrow it	Strong architectural guardrails, matches "events → business logic → new state" mental model	Very productive but easy to bypass MVVM if undisciplined

2.3 Recommendation for this project

Given the user's stated priorities (maintainability, discipline, minimal AI hand-holding) and a server-side background:

Primary recommendation: Bloc + Cubit (via `flutter_bloc`), with `freezed` for immutable states and events.

Reasons:

- Bloc's layer model is literally identical to the Flutter architecture guide's layers** (Presentation → Business Logic → Data → Repository → Data Provider). [6](#)
A developer following `docs.flutter.dev` and a developer following `bloclibrary.dev` converge on the same file layout.
- Bloc/Cubit enforces the discipline that `ChangeNotifier` only encourages** — you cannot accidentally leak business logic into a widget because the widget only emits events and reads immutable states. This directly addresses the "the code should not be mixed" requirement.

3. **The Very Good CLI core template uses Bloc for exactly this reason** — its authors (Very Good Ventures) built it after several years of shipping large Flutter apps, and they picked Bloc for "a clear separation between business logic and presentation". [4](#) The Flutter architecture recommendations page explicitly links to `very_good_cli` as a template that generates a "similar app structure" to the Compass one. [14](#)
4. **Testing is trivial:** `bloc_test` lets you assert "given events *E1, E2* the Bloc emits states *S1, S2*", which matches how a server-side engineer already thinks. [24](#)

***Acceptable alternative: Riverpod 3 with code generation** — pick this if you strongly prefer minimal boilerplate over rigid structure, and you commit to always modeling every feature as a `ViewModel`-shaped Notifier.*

***Do not pick:** raw `Provider` + `ChangeNotifier` for anything beyond the smallest project. It works for the Compass sample, but doesn't give you the compile-time guarantees you need when working with AI-generated code that you don't want to hand-review every time.*

3. Feature-First vs. Layer-First: Resolving the Structure Debate

Andrea Bizzotto's widely referenced analysis on Code With Andrea concludes that **layer-first collapses under scale** because files that belong together end up far apart, while a naïve **feature-first-by-UI-screen** breaks when models are shared between screens. [23](#)

The Compass structure and Bizzotto's refined recommendation converge on the same answer:

- **Group by functional area (domain concept), not by screen.** A "feature" is what the user *does* (authenticate, book, review), not what they see. [23](#)

- **Inside each feature, keep small folders for the four layers** (`presentation` / `application` / `domain` / `data`), or use the Compass split (`ui/<feature>` for the presentation + view-model, and a shared `data/` at the top level). [23](#)
- **Use top-level folders for cross-cutting concerns:** `common_widgets/` , `constants/` , `exceptions/` , `localization/` , `routing/` , `utils/` . [23](#)

The Compass layout adopted in this report is the pragmatic middle: features under `ui/` (because they only own view + view-model), and repositories/services shared under `data/` (because they are used across features and must remain the SSOT). This is exactly the balance the official case study defends. [17](#)

4. Development Tooling and Governance

4.1 Linting and static analysis

- **flutter_lints** (from `flutter.dev`) is the official baseline set of lint rules; it's added to every project generated by `flutter create` . Enable it in `analysis_options.yaml` . It reflects "recommended good coding practices" for Flutter apps, packages, and plugins.
- **very_good_analysis** (from Very Good Ventures) is the stricter, opinionated superset used internally at VGV; the Flutter team's own recommendations page links to it as a resource. [14](#) Choosing this over `flutter_lints` is a good decision for a solo developer who wants stronger safety nets.

4.2 Code generation

The Flutter recommendations page **explicitly recommends** `freezed` or `built_value` for generating immutable data models with JSON serialization, deep equality, and copy methods. [14](#) In practice, combine:

- `freezed` — immutable classes, unions/sealed classes, deep equality, `copyWith` .
- `json_serializable` — JSON (de)serialization.
- `build_runner` — the runner used by all `@`-annotation-based generators.

These same tools are used to generate Bloc events/states and Riverpod providers if you go that route.

4.3 Navigation

The recommendations page states: "**go_router is the preferred way to write 90% of Flutter applications**". [14](#) It's a Flutter-team package, supports declarative URL-based routing, deep links, and — crucially for the user's targets — makes Flutter Web URLs behave like a real web app.

4.4 Testing

`docs.flutter.dev` 's testing guide organizes tests into three levels, all first-party:

- **Unit tests** — for services, repositories, ViewModels/Blocs (the "Strongly recommend: unit-test every one" rule). [14](#)
- **Widget tests** — for views; routing and DI setup are the highest-value spots to test. [14](#)
- **Integration tests** — end-to-end, run against the actual app; the Compass repo has two flavors (local data and staging server) demonstrating this pattern. [12](#)

Pair with:

- mocktail or mockito for test doubles.
- bloc_test when using Bloc.
- **A dedicated `testing/` sub-package**, exactly as the Compass sample does, to expose shared fakes to any package's tests — the "make fakes for testing" rule is one of the "Strongly recommend" items. [17](#) [14](#)

4.5 CI/CD and coverage

- Enforce coverage in CI (VGV's default is 100% line coverage for the core template). [4](#)
- Use the Very Good CLI's `--fail-fast` and `--exclude-coverage` flags if you adopt VGV tooling. [22](#)

4.6 Governance rules to encode from day one

1. One repository = one data type

Every domain concept has exactly one Repository class; nothing else may mutate that data.

2. No cross-repository awareness

Repositories don't know each other; cross-repository composition happens in a use-case or ViewModel.

3. Constructor injection only

All dependencies are private, final fields set via the constructor. No global lookups from inside widgets.

4. Views hold no business logic

Views may only contain simple conditionals, animation, layout-by-size, and simple routing.

5. Abstract Repository interfaces

Every Repository has an abstract interface and one or more concrete implementations (dev / staging / prod), enabling flavor swaps.

6. Immutable data end-to-end

Domain models are **frozen** classes; UI state is an immutable value emitted by the Bloc/Cubit.

5. AI-Assisted Development: The Three Officially Supported Channels

This is the single biggest change from "vibe-coding with Claude" to "AI that stops re-inventing the architecture". The Flutter team ships **three complementary AI mechanisms** on docs.flutter.dev/ai.

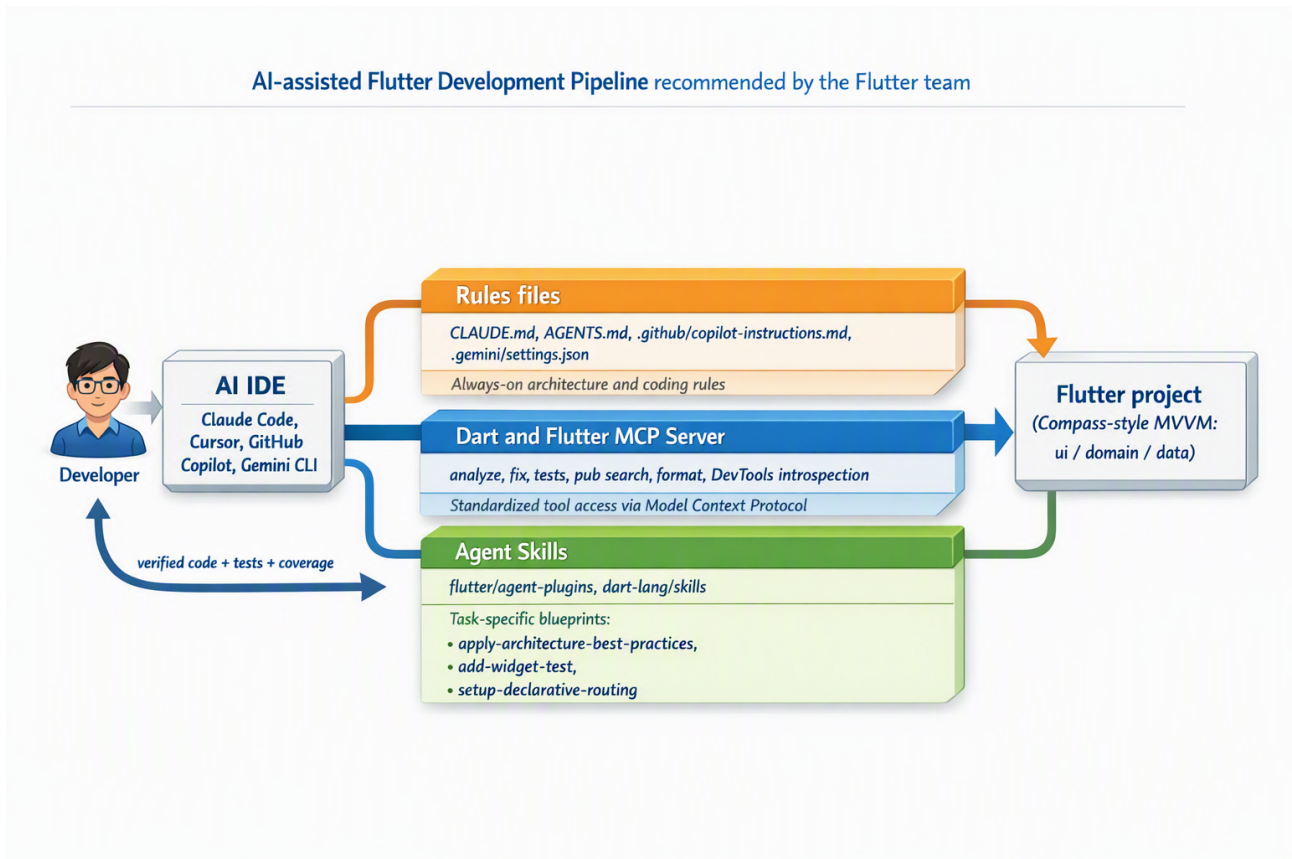


Figure 3: The three complementary AI channels officially supported by the Flutter team — persistent rules files, the Dart and Flutter MCP server, and task-specific Agent Skills — that together let AI assistants respect the project architecture without repeated manual correction.

5.1 Channel 1: Rules files (always-on architecture context)

docs.flutter.dev/ai/ai-rules hosts the official rules.md template that codifies Flutter/Dart best practices. It ships in four sizes (comprehensive, 10k, 4k, 1k) so it fits any tool's context limit. [10](#)

The docs give a per-tool file name convention (last updated 2026-01-05): [10](#)

Tool	Rules file location
Antigravity (Google)	<code>.agent/rules/<rule-name>.md</code>
Claude Code	<code>CLAUDE.md</code>
Cursor	<code>AGENTS.md</code>

Tool	Rules file location
Gemini CLI	<code>GEMINI.md</code>
GitHub Copilot	<code>.github/copilot-instructions.md</code> (~4k chars)
JetBrains AI (Junie)	<code>.junie/guidelines.md</code>
VS Code	<code>.instructions.md</code>

The workflow is: **download rules.md, ask an LLM to convert it to your editor's format, drop it into your repo, verify.** ¹⁰ This should be the very first commit in your new project — everything the AI does afterward is grounded by it.

5.2 Channel 2: The Dart and Flutter MCP Server (tools)

The **Dart and Flutter MCP server** (currently experimental) exposes a growing list of Dart/Flutter tools to AI assistants via the Model Context Protocol. ⁹ It lets the AI: ⁹

- Analyze and fix errors in your code.
- Resolve symbols, fetch documentation and signatures.
- Introspect and interact with your running app.
- Search `pub.dev` for the best package.
- Manage `pubspec.yaml` dependencies.
- Run tests and analyze results.
- Format code with the same formatter as `dart format`.

Configuration is a single JSON entry (`"command": "dart", "args": ["mcp-server"]`) added to Antigravity, Gemini CLI, Cursor, or GitHub Copilot (in VS Code, via the Dart Code extension v3.116+). ⁹

5.3 Channel 3: Agent Skills (task-specific blueprints)

`docs.flutter.dev/ai/agent-skills` introduces **Agent Skills**: task-oriented blueprints the AI reads only when needed (progressive disclosure). ⁸ The Flutter team hosts them at `flutter/agent-plugins` (previously named `flutter/skills`). ⁸

The current skill catalog directly addresses the exact things you'd otherwise have to keep correcting: [21](#)

- flutter-apply-architecture-best-practices
- flutter-add-widget-test
- flutter-add-integration-test
- flutter-add-widget-preview
- flutter-build-responsive-layout
- flutter-implement-json-serialization
- flutter-setup-declarative-routing
- flutter-setup-localization
- flutter-use-http-package
- flutter-fix-layout-issues

Installation:

`npx skills add flutter/agent-plugins --skill '*' --agent universal`. [8](#) A parallel dart-lang/skills repo covers Dart tasks (unit tests, dependency resolution, static analysis fixes). [8](#)

5.4 How the three channels work together

- **Rules** are read on every request → they hold the invariants ("use MVVM; do not put logic in widgets; use go_router; use freezed").
- **MCP** is invoked as tools during a request → it runs `dart analyze`, `dart format`, `flutter test`, `pub add`, etc., so the AI verifies its own work.
- **Skills** are read only when a specific task starts → they hold the step-by-step procedure ("to add a widget test, do X, Y, Z").

Together, they replace the "constant deep-review feedback loop" the user described.

5.5 Complementary MCP servers worth adding

Very Good Ventures' engineering team lists seven more MCP servers useful for Dart/Flutter dev: [3](#) Git, GitHub, Atlassian (Jira/Confluence), Figma, iOS Simulator, and a generic Fetch server. For a solo developer the **Git + GitHub** pair is the highest ROI — the AI can then read issues, branch, commit, and open PRs without you switching context.

VGV also ship their own MCP server inside `very_good_cli 1.0.0`, which lets AI agents run VGV CLI scaffolding commands from Claude, Cursor, or any MCP-compatible tool. [22](#)

6. Templates and Starters You Can Actually Trust

Compass sample app — reference architecture

Full-featured travel-booking app maintained by the Flutter team, used verbatim in the official architecture case study. Demonstrates development + staging environments, real HTTP server, high test coverage. The recommended starting point if you want to *read* production Flutter code.

Very Good CLI (`very_good_cli 1.0.0`) — opinionated template generator

Generates a Flutter app with Bloc, Very Good Analysis, three flavors (dev/staging/prod), i18n, 100% test coverage baseline, GitHub Actions CI, and now an integrated MCP server. Recommended by the Flutter architecture page as a template that produces "a similar app structure" to Compass.

`flutter/agent-plugins` — Agent Skills

Official Flutter Agent Skills, including the crucial `flutter-apply-architecture-best-practices`. Install once, use forever.

Flutter's `rules.md` — AI baseline rules

Four sizes to fit any AI tool; convert to `CLAUDE.md`/`AGENTS.md`/`copilot-instructions.md` as appropriate.

[12](#) [4](#) [22](#) [14](#) [21](#) [10](#)

Recommended combination for this project: generate the shell with `very_good create flutter_app` for the CI/analysis/flavors baseline, then reorganize `lib/` to mirror the Compass folder layout (adding `data/`, `domain/`, `ui/core/`, `ui/<feature>/{view_models,widgets}`), then install the AI rules and Flutter/Dart Agent Skills.

7. Matching FastAPI Architecture

Your Flutter clarity is worth nothing if the backend is a swamp. The Flutter feature-first idea translates almost verbatim to FastAPI.

7.1 What the FastAPI docs actually recommend

The official "**Bigger Applications — Multiple Files**" tutorial demonstrates splitting an app across a small tree, using `APIRouter` to keep routes modular: [2](#)

```
app/
├── __init__.py
├── main.py
├── dependencies.py
├── routers/
│   ├── __init__.py
│   ├── items.py
│   └── users.py
└── internal/
    ├── __init__.py
    └── admin.py
```

This is the smallest correct structure. It stops working as soon as you have more than a couple of domains.

7.2 The consensus scalable structure

The most widely cited scalable layout is Zhanyim Kanov's `zhanyimkanov/fastapi-best-practices` (17k+ stars, inspired by Netflix's Dispatch project). Its per-domain module holds *everything* that domain needs: [1](#)

```
fastapi-project
├── alembic/
├── src/
│   ├── auth/
│   │   ├── router.py      # all endpoints for this domain
│   │   ├── schemas.py    # Pydantic models (DTOs)
│   │   ├── models.py     # DB models
│   │   ├── dependencies.py # router dependencies (auth, pagination, ...)
│   │   ├── config.py     # local env config
│   │   ├── constants.py  # error codes, magic strings
│   │   ├── exceptions.py  # domain-specific exceptions
│   │   ├── service.py    # business logic
│   │   └── utils.py      # non-business helpers
│   ├── posts/            # same layout
│   ├── aws/              # 3rd-party integrations
│   ├── config.py         # global config
│   └── models.py         # global DB base
```

```

|   ├── exceptions.py      # global exceptions
|   ├── pagination.py      # shared cross-domain helpers
|   ├── database.py        # DB connection
|   └── main.py            # FastAPI app factory
└── tests/
    ├── auth/
    ├── aws/
    └── posts/
└── requirements/
    ├── base.txt
    ├── dev.txt
    └── prod.txt
└── .env
└── logging.ini
└── alembic.ini

```

Cross-domain imports use explicit module names: [1](#)

```

from src.auth import constants as auth_constants
from src.notifications import service as notification_service
from src.posts.constants import ErrorCode as PostsErrorCode

```

7.3 How this maps to your Flutter side

Flutter concept	FastAPI concept
<code>ui/<feature>;</code>	<code>src/<domain>/router.py</code> (one API surface per feature)
<code>data/services/ <domain>_api_client.dart</code>	<code>src/<domain>/router.py</code> (endpoints) + <code>src/<domain>/schemas.py</code> (contract)
<code>data/repositories/ <domain>_repository.dart</code>	<code>src/<domain>/service.py</code> (business logic)
<code>domain/models/<domain>.dart</code>	<code>src/<domain>/schemas.py</code>
<code>test/data/...</code>	<code>tests/<domain>/...</code>

Both sides share the same "one folder per business capability, everything for that capability inside" mental model. That symmetry is the single biggest reason to insist on this layout — a change to the "bookings" domain touches one folder in Flutter and one folder in FastAPI.

7.4 Additional FastAPI hard rules worth borrowing

From the same conventions repo: [1](#)

- **Async routes for I/O**; use sync routes (or a thread pool) only for blocking I/O; offload CPU-bound work to Celery/workers.
- **Excessively use Pydantic** — validate every request/response, share a custom base model.
- **Decouple `pydantic-settings`** — one `Settings` class per module for local config, plus a global one.
- **Dependency injection with `Depends()`** — chain dependencies, reuse them, prefer async, and use `dependency_overrides` in tests.
- **Alembic migrations from day one.**
- **SQL-first schemas, Pydantic-second** — model DB, then adapt Pydantic to it.
- Use `ruff` for formatting/linting.
- **Set up async test client from day zero.**

7.5 CORS, because Web is a target

The FastAPI CORS docs are the canonical reference: enable the `CORSMiddleware` in `main.py` and whitelist your Flutter Web origin (localhost during dev, your deployed origin in staging/prod). This is trivially forgettable and blocks all cross-origin requests silently — put it in the rules file so the AI never forgets.

8. Migration: New Project vs. Refactor the Existing One

The user has a working app and is willing to either refactor or start over. Two paths, one decision framework.

8.1 Path A — Incremental refactor of the current app

Feasible only if the current app already:

- Compiles on Flutter 3.x with null safety.
- Has any existing tests (even a few).
- Uses `go_router` or Navigator 2.0 (not raw Navigator 1.0).
- Doesn't mix mutable global singletons everywhere.

Steps:

1. **Freeze features.** No new features during the refactor.
2. **Drop in the tooling:** `flutter_lints` (or `very_good_analysis`), `analysis_options.yaml`, `freezed`, `json_serializable`, `build_runner`, `go_router`, `flutter_bloc`, `bloc_test`, `mocktail`, `equatable`.
3. **Add the AI rules + MCP + skills** so the AI is architecture-aware before you touch code.
4. **Refactor by extraction, feature by feature:**
 - Extract API calls into a stateless Service.
 - Extract business logic and caching into a Repository.
 - Rewrite the widget as a View + Bloc/Cubit pair.
 - Add unit tests for the Repository, Bloc, and widget tests for the View.
5. **Restructure folders last**, once each feature is already MVVM-shaped.

8.2 Path B — Greenfield project, port feature-by-feature

Steps:

1. `very_good create flutter_app my_app` (or `flutter create` if you want a lighter skeleton).
2. Immediately reshape `lib/` into `config/`, `data/`, `domain/`, `ui/{core, <feature>}`, `routing/`, `utils/`.
3. Add `CLAUDE.md` (or your tool's equivalent) generated from Flutter's `rules.md`, install the MCP server and Agent Skills.
4. Set up three flavored entry points (`main_development.dart`, `main_staging.dart`, `main_production.dart`).

5. **Port one feature at a time** from the old app: for each, tell the AI to (a) read the old feature, (b) create a new feature module using the `flutter-apply-architecture-best-practices` skill, (c) write tests, (d) `flutter test`.
6. Retire the old app once every feature has moved.

8.3 Decision criteria

Choose Path A (refactor) if...

Fewer than ~10 screens; the existing app already uses `go_router`; you have any test coverage; codebase is under active production use and can't be paused.

Choose Path B (greenfield + port) if...

Existing app is a prototype; state management is inconsistent; you have zero tests; you want to be able to justify the new architecture to yourself with a clean slate. This is almost always the right choice for a first serious project.

Given the user's context (this is their *first* application, they hit a dead end with AI-generated code, they want a clean structure), **Path B is the recommended path**. Use the existing app as a *specification* — feed it to the AI as reference and have it re-implement each feature under the new architecture.

9. The Blueprint (Copy-Paste Ready)

9.1 Flutter project — folder tree

```
my_app/
├─ analysis_options.yaml      # include: package:very_good_analysis/analysis_options.yaml
├─ pubspec.yaml
├─ CLAUDE.md                  # AI rules (converted from Flutter's rules.md)
├─ AGENTS.md                  # (if using Cursor / universal AGENTS.md)
├─ .github/
│   └─ copilot-instructions.md # 4k version of rules.md
├─ .agents/
│   └─ skills/                 # installed via npx skills add flutter/agent-plugins
├─ lib/
│   └─ main.dart
│   └─ main_development.dart
│   └─ main_staging.dart
│   └─ main_production.dart
```

```

|   ├── config/
|   |   ├── app_config.dart          # env-specific settings loaded per entry point
|   |   └── dependencies.dart        # MultiProvider / GetIt setup
|   ├── routing/
|   |   ├── router.dart              # go_router configuration
|   |   └── routes.dart              # route path constants
|   ├── data/
|   |   ├── services/
|   |   |   ├── api/
|   |   |   |   ├── api_client.dart    # wraps FastAPI base URL + Dio
|   |   |   |   └── model/              # raw JSON models (freezed)
|   |   |   └── local/
|   |   |       └── shared_prefs_service.dart
|   |   └── repositories/
|   |       ├── auth/
|   |       |   ├── auth_repository.dart    # abstract
|   |       |   ├── auth_repository_remote.dart
|   |       |   └── auth_repository_dev.dart # local JSON for dev flavor
|   |       └── bookings/
|   |           ├── bookings_repository.dart
|   |           └── bookings_repository_remote.dart
|   ├── domain/
|   |   ├── models/
|   |   |   ├── user.dart              # freezed domain model
|   |   |   └── booking.dart
|   ├── ui/
|   |   ├── core/
|   |   |   ├── ui/                   # branded buttons, error banners, ...
|   |   |   └── themes/               # Material 3 ThemeData
|   |   ├── auth/
|   |   |   ├── view_models/
|   |   |   |   └── login_cubit.dart
|   |   |   └── widgets/
|   |   |       └── login_screen.dart
|   |   └── home/
|   |       ├── view_models/
|   |       |   └── home_bloc.dart
|   |       └── widgets/
|   |           ├── home_screen.dart
|   |           └── home_booking_tile.dart
|   └── utils/
|       ├── result.dart              # Result<T,E> for error handling
|       └── command.dart             # Command pattern helper
└── test/
    ├── data/
    ├── domain/
    ├── ui/
    └── testing/
        ├── fakes/                  # fake repositories used by any test package
        └── models/                 # canned domain-model fixtures

```

9.2 pubspec.yaml — recommended dependencies

```
name: my_app
description: My first cross-platform app.

environment:
  sdk: ^3.7.0
  flutter: ">=3.35.0"

dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter
  flutter_bloc: ^9.0.0          # primary state management
  freezed_annotation: ^3.0.0   # immutable states / events / domain models
  json_annotation: ^4.9.0
  go_router: ^17.0.0           # official routing package
  dio: ^5.4.0                   # HTTP client (or plain http package)
  shared_preferences: ^2.3.0    # key-value store
  equatable: ^2.0.5

dev_dependencies:
  flutter_test:
    sdk: flutter
  bloc_test: ^10.0.0           # bloc test helpers
  mocktail: ^1.0.0
  build_runner: ^2.4.0         # code generation runner
  freezed: ^3.0.0
  json_serializable: ^6.7.0
  very_good_analysis: ^10.0.0 # opinionated lint rules (or flutter_lints)

flutter:
  uses-material-design: true
  generate: true               # for l10n
```

9.3 FastAPI project — folder tree

```
backend/
├─ pyproject.toml
├─ alembic.ini
├─ logging.ini
├─ .env
├─ AGENTS.md          # zhanymkanov's AGENTS.md, tailored
├─ alembic/
├─ src/
│   ├─ main.py         # FastAPI app factory + CORS + routers
│   ├─ config.py       # global Settings (pydantic-settings)
│   ├─ database.py     # engine, session factory
│   ├─ models.py       # SQLAlchemy Base
│   ├─ exceptions.py   # global exceptions
│   └─ pagination.py
```

```

|   └─ auth/
|   |   └─ router.py
|   |   └─ schemas.py
|   |   └─ models.py
|   |   └─ dependencies.py
|   |   └─ service.py
|   |   └─ constants.py
|   |   └─ exceptions.py
|   |   └─ config.py
|   └─ bookings/          # one folder per domain
|       └─ (same layout)
└─ tests/
    └─ auth/
    └─ bookings/
└─ requirements/
    └─ base.txt
    └─ dev.txt
    └─ prod.txt

```

9.4 The one-page architecture cheat sheet

Dependencies flow inward only

View → ViewModel → (Use Case) → Repository → Service.
Nothing points the other way.
Compile-checked by making injected fields `final`.

One folder per business capability

Both in Flutter's `ui/<feature>/` and FastAPI's `src/<domain>/`.
Cross-cutting code goes to `ui/core/` or `src/` root.

Everything immutable

Domain models: `freezed`.
States/events: `freezed`.
Pydantic models: `frozen=True` where possible.

Abstract every Repository

Interface + real impl + fake impl. Dev flavor uses fake; test suite reuses the same fake from the `testing/` package.

Three entry points from day one

`main_development.dart` ,
`main_staging.dart` ,
`main_production.dart` — set
up DI, base URL, and logging
per flavor.

AI channels wired on day one

Rules file + Dart & Flutter MCP
server + Flutter Agent Skills
installed before the first feature
is written.

9.5 A "Day-One Setup" checklist

1. Install Flutter 3.35+ / Dart 3.9+ and enable Web/iOS/Android targets.
 2. `dart pub global activate very_good_cli` → `very_good create flutter_app my_app` (or `flutter create`).
 3. Add `very_good_analysis` (or `flutter_lints`) in `analysis_options.yaml`.
 4. Reshape `lib/` per §9.1.
 5. Add dependencies from §9.2.
 6. Download `rules.md` from `docs.flutter.dev/ai/ai-rules`, convert to your editor's rules file, commit. [10](#)
 7. Configure the **Dart and Flutter MCP server** in your AI IDE (`"command": "dart", "args": ["mcp-server"]`). [9](#)
 8. Install Agent Skills: `npx skills add flutter/agent-plugins --skill '*' --agent universal`. [8](#)
 9. Add Git + GitHub MCP servers to your AI client. [3](#)
 10. Bootstrap CI (GitHub Actions) running `flutter analyze`, `flutter test --coverage`, and `dart format --set-exit-if-changed .`
 11. Scaffold the FastAPI backend per §9.3.
 12. Add CORS for `http://localhost:*` and your deployed Flutter Web origin.
 13. Write one end-to-end feature (login) as the *reference feature* and commit it. Every future feature copies its shape.
-

10. What This Report Deliberately Does Not Do

- **It does not invent a Clean-Architecture variant.** Multiple articles describe layered Clean Architecture with UseCases, Entities, Data Sources, etc. The Flutter team's guide covers 90% of that value with less overhead and a "conditional" use-case layer you can add when you actually need it. Sticking with the official guide keeps you compatible with `flutter/agent-plugins` skills and community tooling.
 - **It does not push Riverpod as the default.** Riverpod 3 is a great fit, but its provider graph is a different mental model from the MVVM guide; Bloc/Cubit's event → new state loop is a smaller conceptual step from server-side handlers to Flutter UI.
 - **It does not adopt a Melos monorepo yet.** Melos is powerful for multi-package modularization, but for a first project a single package with a Compass-style layout is sufficient. Revisit once you have shared libraries across multiple apps.
-

11. Reference Reading List (all official or near-official)

- **Flutter architecture guide** — the "Architecting Flutter apps" section on `docs.flutter.dev/app-architecture` (concepts, guide, case study, dependency injection, data layer, recommendations, design patterns). [20](#) [19](#) [18](#) [17](#) [16](#) [15](#) [14](#) [13](#)
- **Compass sample app** — `flutter/samples/compass_app`: a full-feature reference implementation. [12](#) [11](#)
- **Flutter AI docs** — rules, MCP server, and agent skills on `docs.flutter.dev/ai`. [10](#) [9](#) [8](#)
- **State management options** — `docs.flutter.dev/data-and-backend/state-mgmt/options`. [7](#)
- **Bloc library** — `bloclibrary.dev/architecture` for the layered model. [6](#)
- **Riverpod** — `riverpod.dev` and its "About code generation" page (if you go Riverpod). [5](#)
- **Very Good CLI** — `cli.vgv.dev` and Very Good Analysis (`pub.dev/packages/very_good_analysis`). [4](#)
- **7 MCP Servers Every Dart and Flutter Developer Should Know** — VGV's curated list for a broader AI toolchain. [3](#)
- **FastAPI Bigger Applications** — the official multi-file tutorial. [2](#)

- **zhanymkanov/fastapi-best-practices** — the community-standard scalable structure and conventions. [1](#)

Follow this blueprint and there is no architectural decision left for the AI to invent on its own — every request it receives is bounded by the rules file, executed with the MCP tools, and shaped by the Agent Skill for the specific task. That is what turns Claude Code (or Cursor, Copilot, Gemini CLI) from a source of "deep-review feedback loops" into a reliable pair-programmer that respects the architecture you chose.